NODE_ID_DRSW Data Flow

How `d_node_ids` and `d_node_offsets` are read and written across BFS depths

01 GPU Arrays

d_node_ids
int[num_rows] — ~11M entries
PERSISTENT

d_node_offsets
int[num_rows] — ~11M entries
REBUILT EACH DEPTH

d_selected_examples
int[total_rows] — ~7M entries
REBUILT EACH DEPTH

d_node_row_off
int[num_nodes + 1]
HOST PREFIX SUM

`d_node_ids` is the only array that carries state from one BFS depth to the next. Every other array is derived from it.

02 SRDW Projected Value Layout

SHARED INDEXING FORMULA (ALL KERNELS)

```
projected[row_start * num_proj + proj_id * rows_node + local_r]
```

`row_start = d_node_row_off[nid]` · `rows_node = d_node_row_off[nid+1] - row_start` · `local_r = d_node_offsets[row]`

The `projected[]` buffer is dense: all rows of node *n* for projection *p* are contiguous. Both the Projection kernel (write) and SplitUpdate kernel (read) use `d_node_offsets[row]` as the `local_r` index.

03 Per-Depth Pipeline

Init

Depth 1 only. Sets `d_node_ids` from bootstrap `d_selected_examples`.

WRITE `d_node_ids` READ `d_selected_examples`, `d_node_row_off`

Memsets all `d_node_ids` to −1, then for each position in `d_selected_examples` finds the owning node via `cub::UpperBound`. This is the only CUB UpperBound call in NODE_ID_DRSW.

ComputeOffsets

Every depth. Single pass over `num_rows` derives offsets and rebuilds `d_selected_examples`.

READ `d_node_ids`, `d_node_row_off` WRITE `d_node_offsets`, `d_selected_examples`

Warp-aggregated atomics: `__match_any_sync` groups lanes with the same `node_id`, leader does one `atomicAdd` per group (~32× fewer atomics), `__shfl_sync` broadcasts the base offset. Each thread writes two arrays:

```
node_offsets[row] = local_r
d_selected_examples[row_start + local_r] = row
```

Projection

Every depth. Dot products → `projected[]` in SRDW layout.

READ `d_node_ids`, `d_node_offsets`, `d_node_row_off` WRITE `projected[]`

Reuses the existing `ColumnAddProjectionKernel_DRSW_1` kernel. Iterates all `num_rows`, reads `node_ids[r]` and `node_offsets[r]` to compute the write index. NODE_ID_DRSW skips the `ComputeDRSWArrays` setup kernel entirely.

MinMax

Every depth. Min/max of projected values per node.

READ `projected[]` NONE `d_node_ids`, `d_node_offsets`

Reads contiguous blocks in the SRDW layout sequentially. No indirection through `node_ids` or `node_offsets` needed.

Histogram

Every depth. Bin projected values, accumulate class counts.

READ `projected[]`, `d_selected_examples`, `d_labels` NONE `d_node_ids`, `d_node_offsets`

Uses `d_selected_examples[row_start + r]` as the inverse mapping: SRDW offset → dataset row → class label. This is **why ComputeOffsets must rebuild** `d_selected_examples` each depth.

SplitUpdate

Every depth. Compare projected vs threshold, rewrite `node_ids` in place.

READ `d_node_ids`, `d_node_offsets`, `projected[]` WRITE `d_node_ids` *(in-place)*

Direct lookup: `nid = node_ids[row]` (no CUB UpperBound). Reads the projected value via `node_offsets[row]`, compares against threshold, writes `node_ids[row] = child_id`. Propagates state to the next depth.

04 Key Code Paths

ComputeOffsets — warp-aggregated atomics

node_id_drsw.cuh:78–115

```
int nid = node_ids[row]; // Group warp lanes sharing the same node_id unsigned match_mask = __match_any_sync(active, nid); int leader = __ffs(match_mask) - 1; int group_size = __popc(match_mask); // One atomic per warp-group instead of per thread if (lane == leader) base_offset = atomicAdd(&d_atomic_counters[nid], group_size); base_offset = __shfl_sync(match_mask, base_offset, leader); // Per-thread position within group int local_r = base_offset + __popc(match_mask & lower_mask); // Dual write: offset for projection + inverse map for histogram node_offsets[row] = local_r; d_selected_examples[row_start + local_r] = row;
```

Projection — SRDW layout write

oblique_apply_projection.cu:2022–2067

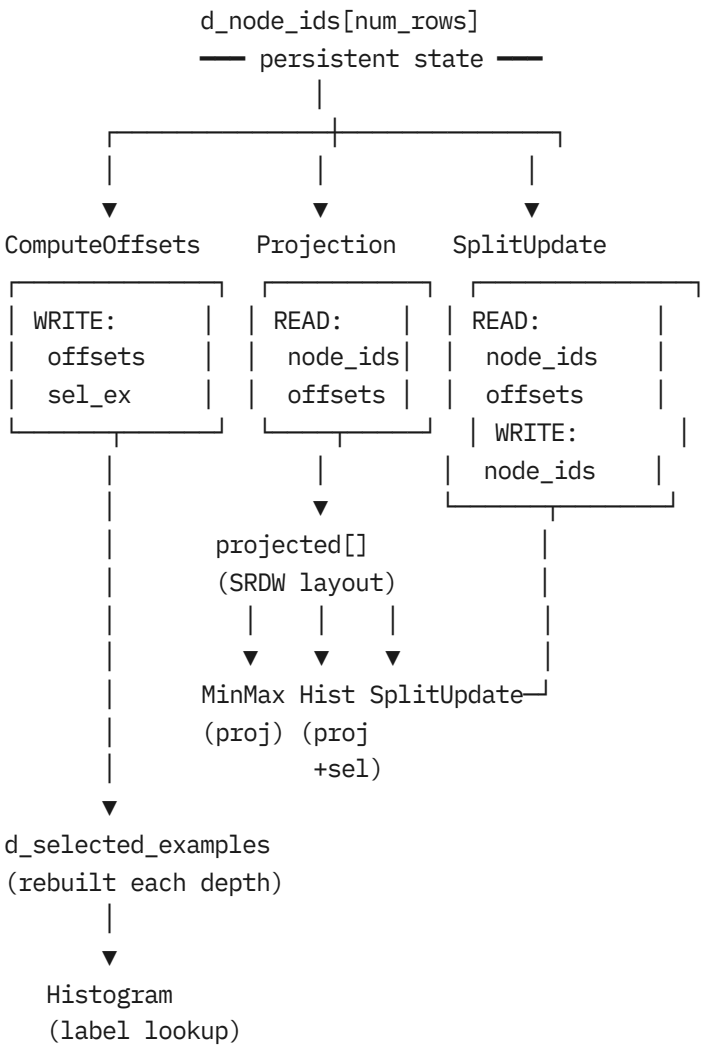
```
int node_id = node_ids[r]; int node_offset = node_offsets[r]; int row_start = d_node_row_off[node_id]; int rows_node = d_node_row_off[node_id + 1] - row_start; // Dot product → dense SRDW layout projected[(long long)row_start * num_proj + (long long)proj_id * rows_node + node_offset] = sum;
```

SplitUpdate — direct node_ids lookup

node_id_drsw.cuh:173–198

```
int nid = node_ids[row]; // direct — no CUB UpperBound int local_r = node_offsets[row]; long long base = (long long)row_start * num_proj + (long long)best_proj * rows_node; float val = projected[base + local_r]; int child_id = (val >= threshold) ? pos_child : neg_child; node_ids[row] = child_id; // in-place update → next depth
```

05 Data Flow Diagram



Why rebuild `d_selected_examples`? The Histogram kernel needs an inverse mapping from SRDW offset to dataset row: `example_idx = d_selected_examples[row_start + r]` to look up `d_labels[example_idx]`. Without it, there is no path from a projected value's position back to its class label.

06 Baseline DRSW vs NODE_ID_DRSW

Aspect	Baseline DRSW	NODE_ID_DRSW
Persistent state	`d_selected_examples` (sorted, partitioned)	`d_node_ids[num_rows]`
Per-depth setup	ComputeDRSWArrays CUB UpperBound over total_rows	ComputeOffsets Warp-aggregated atomics over

Aspect	Baseline DRSW	NODE_ID_DRSW
		num_rows
SplitUpdate lookup	CUB UpperBound per thread	Direct `node_ids[row]` read
Partition step	CUB RadixSort or fused scan+scatter	None — `node_ids` updated in place
d_selected_examples	Maintained as sorted partition	Rebuilt as side effect of ComputeOffsets

07 Performance



Breakdown: apply_proj 75→63ms (skip ComputeDRSWArrays), partition 50→37ms (no UpperBound), new ComputeOffsets ~17ms. Net: **14ms faster** (−5%).